

Sir Syed University of Engineering & Technology

Computer Science & Information Technology Department

Intro. to Operating System (CS-222T) — Assignment #2 (Solved)

Question #01 — Peterson's Algorithm

Write the code of Peterson's Algorithm.

Peterson's Algorithm is a classic software-based mutual exclusion solution for two processes. It uses two shared variables — a **flag[]** array (indicating intent to enter critical section) and a **turn** variable (indicating whose turn it is) — to ensure that only one process enters the critical section at a time, satisfying mutual exclusion, progress, and bounded waiting.

C Code:

```
#include <stdio.h>
#include <pthread.h>
#include <stdbool.h>

// Shared variables
int turn;
bool flag[2];

void enter_critical(int i) {
    int j = 1 - i; // the other process
    flag[i] = true; // express intent to enter
    turn = j; // give turn to the other
    // Busy-wait: enter only when other is not interested OR it is our turn
    while (flag[j] == true && turn == j);
}

void exit_critical(int i) {
    flag[i] = false; // release the critical section
}

void *process(void *arg) {
    int id = *(int *)arg;
    enter_critical(id);
    printf("Process %d in critical section\n", id);
    exit_critical(id);
    printf("Process %d exited critical section\n", id);
    return NULL;
}

int main() {
    pthread_t t0, t1;
    int id0 = 0, id1 = 1;
    flag[0] = flag[1] = false;
    pthread_create(&t0, NULL, process, &id0);
    pthread_create(&t1, NULL, process, &id1);
    pthread_join(t0, NULL);
```

```
pthread_join(t1, NULL);
return 0;
}
```

How it works: Before entering the critical section, a process sets its flag to true and yields the turn to the other. It then busy-waits as long as the other process also wants to enter AND it is still the other's turn. This ensures mutual exclusion — both processes can never be inside simultaneously — while also guaranteeing progress and bounded waiting (at most one wait cycle).

Question #02 — Bakery Algorithm

Write the code of Bakery Algorithm.

Lamport's Bakery Algorithm generalises Peterson's solution to **N processes**. Each process takes a numbered 'ticket' (like customers in a bakery); the process with the smallest ticket number enters the critical section next. Ties are broken by process ID. It uses a **choosing[]** array (to signal ticket selection in progress) and a **number[]** array (the ticket values). It satisfies mutual exclusion, progress, and bounded waiting for any N.

C Code (for N processes):

```
#include <stdio.h>
#include <pthread.h>
#include <stdbool.h>
#include <string.h>

#define N 5 // number of processes

int number[N]; // ticket numbers
bool choosing[N]; // is process i choosing a number?

// Returns max value in the number[] array
int max_number() {
    int m = 0;
    for (int i = 0; i < N; i++) if (number[i] > m) m = number[i];
    return m;
}

void lock(int i) {
    choosing[i] = true;
    number[i] = 1 + max_number(); // take a ticket
    choosing[i] = false;

    for (int j = 0; j < N; j++) {
        // Wait while j is choosing its number
        while (choosing[j]);
        // Wait while j has a smaller (or equal with lower id) ticket
        while (number[j] != 0 &&
            (number[j] < number[i] ||
            (number[j] == number[i] && j < i)));
    }
}
```

```

void unlock(int i) {
number[i] = 0; // release ticket
}

void *process(void *arg) {
int id = *(int *)arg;
lock(id);
printf("Process %d in critical section\n", id);
unlock(id);
return NULL;
}

int main() {
pthread_t threads[N];
int ids[N];
memset(number, 0, sizeof(number));
memset(choosing, false, sizeof(choosing));
for (int i = 0; i < N; i++) { ids[i] = i; pthread_create(&threads[i], NULL, process, &ids[i]); }
for (int i = 0; i < N; i++) pthread_join(threads[i], NULL);
return 0;
}

```

How it works: Each process first picks a ticket number one greater than the current maximum. It then waits for all processes that are currently choosing, and for any process with a smaller ticket (or same ticket but lower ID). Once those conditions clear, it enters the critical section and sets its number back to 0 on exit. Because ticket numbers are strictly ordered, starvation is impossible.

Question #03 — Banker's Algorithm (Deadlock Avoidance)

Given: 5 processes (P0–P4), 4 resource types (A, B, C, D). Max: A=3, B=17, C=16, D=12. Find the Need matrix and determine if the system is in a safe state.

Given Matrices (from assignment):

Process	Alloc A	Alloc B	Alloc C	Alloc D	Max A	Max B	Max C	Max D
P0	0	1	1	0	0	2	1	0
P1	1	2	3	1	1	6	5	2
P2	1	3	6	5	2	3	6	6
P3	0	6	3	2	0	6	5	2
P4	0	0	1	4	0	6	5	6
Total	2	12	14	12	—	—	—	—

Available vector: $A = 3 - 2 = 1$, $B = 17 - 12 = 5$, $C = 16 - 14 = 2$, $D = 12 - 12 = 0 \rightarrow \text{Available} = [1, 5, 2, 0]$

Need Matrix (Need = Max – Allocation):

Process	Need A	Need B	Need C	Need D
P0	0	1	0	0

P1	0	4	2	1
P2	1	0	0	1
P3	0	0	2	0
P4	0	6	4	2

Safety Algorithm — Step-by-step:

Step	Process	Need $\leq$ Available?	Work after grant	Finish
1	P0	$[0,1,0,0] \leq [1,5,2,0]$ ✓	$[1,6,3,0]$	True
2	P2	$[1,0,0,1] \leq [1,6,3,0]$ ✓	$[2,9,9,5]$	True
3	P3	$[0,0,2,0] \leq [2,9,9,5]$ ✓	$[2,15,12,7]$	True
4	P1	$[0,4,2,1] \leq [2,15,12,7]$ ✓	$[3,17,15,8]$	True
5	P4	$[0,6,4,2] \leq [3,17,15,8]$ ✓	$[3,17,16,12]$	True

Conclusion: The system IS in a **safe state**. The safe sequence is: **P0** → **P2** → **P3** → **P1** → **P4**. At each step, the process's Need vector does not exceed the current Work (available) vector. After granting resources and the process finishes, its allocated resources are added back to Work, making more resources available for subsequent processes. Since all 5 processes complete, no deadlock can occur in this state.